PROJETO INTEGRADOR II

Sistema de Acompanhamento de Demandas de Desenvolvimento

REUNIÃO 4 – API COM EXPRESS E TYPESCRIPT: ROTAS GET E POST

> **Objetivo da etapa.** Evoluir o servidor básico criado na Reunião 3 para uma API construída com Express e TypeScript, implementando as primeiras rotas GET e POST do sistema. Cada integrante deverá desenvolver as rotas que atendem à tela sob sua responsabilidade. Nesta etapa, os dados ficarão armazenados em memória; a conexão com o banco de dados relacional e a integração entre as telas e a API serão trabalhadas nas próximas etapas.

1. Contexto da Reunião 4

Na Reunião 2, cada integrante desenvolveu sua tela em HTML e CSS. Na Reunião 3, essas telas receberam validações em JavaScript e a equipe configurou o ambiente Node.js com um servidor básico em funcionamento. Agora, o backend deverá começar a responder às operações do sistema, por meio de rotas que consultam (GET) e registram (POST) informações.

As rotas desenvolvidas nesta etapa deverão corresponder às telas já existentes. Assim, cada aluno dá continuidade ao trabalho iniciado nas reuniões anteriores, agora do lado do servidor.

2. Entregas esperadas

2.1. Parte da equipe – estrutura da API com Express e TypeScript

A equipe deverá preparar a estrutura comum do backend, que será utilizada por todos os integrantes.

- utilizar a versão LTS vigente do Node.js com a linguagem TypeScript, conforme definido no Documento de Visão;
- configurar o Express no servidor criado na Reunião 3;
- organizar o backend em arquivos e pastas separando, no mínimo, o ponto de entrada do servidor, as rotas e os dados de teste;
- criar um conjunto de dados de teste em memória (arrays ou arquivo JSON), compartilhado pela equipe, contendo projetos, usuários dos três perfis (Administrador, Líder de Projeto e Membro da Equipe) e demandas com tipos, prioridades, status e prazos variados;
- padronizar as respostas da API no formato JSON;
- utilizar códigos de status HTTP adequados a cada situação, como 200 (sucesso), 201 (registro criado), 400 (dados inválidos), 401 (credenciais inválidas) e 404 (registro não encontrado);
- atualizar o README.md com as instruções para instalar as dependências e executar o servidor.

> **Atenção:** os dados de teste devem ser suficientes para demonstrar as rotas de todos os integrantes, principalmente filtros, buscas, ordenação e os indicadores do dashboard. Quando o banco de dados for integrado, a estrutura das rotas deverá ser mantida, substituindo-se apenas a forma de acesso aos dados.

2.2. Parte individual – rotas GET e POST

Cada integrante deverá implementar as rotas relacionadas à tela sob sua responsabilidade. A sugestão de distribuição, mantendo a divisão das reuniões anteriores, é:

Integrante	Tela (Reuniões 2 e 3)	Rotas sob responsabilidade
Aluno 1	Login	`POST /login` e `GET /usuarios`
Aluno 2	Dashboard	`GET /dashboard`
Aluno 3	Listagem de Demandas	`GET /demandas` (com filtros, busca e ordenação)
Aluno 4	Cadastro/Edição de Demanda	`POST /demandas` e `GET /projetos`
Aluno 5	Detalhes da Demanda	`GET /demandas/:id`, `GET /demandas/:id/comentarios` e `POST /demandas/:id/comentarios`

Observação: o grupo poderá ajustar a distribuição entre os integrantes e os nomes das rotas, mas todos deverão possuir uma responsabilidade individual claramente identificada.

Aluno 1 – Login e usuários

- `POST /login`: receber e-mail e senha, conferir com os usuários cadastrados nos dados de teste e retornar os dados do usuário e seu perfil, sem devolver a senha;
- retornar erro 400 quando houver campos obrigatórios ausentes e erro 401 quando as credenciais forem inválidas;
- `GET /usuarios`: listar os usuários pré-cadastrados, sem expor as senhas.

Aluno 2 – Dashboard

- `GET /dashboard`: retornar o total de demandas e as quantidades por status (Aberta, Em andamento, Em revisão, Concluída e Cancelada), por prioridade e por tipo;
- retornar as demandas críticas em aberto e as demandas próximas do prazo de finalização;
- a equipe deverá definir e explicar o critério adotado para “em aberto” e para “próximas do prazo”.

Aluno 3 – Listagem de demandas

- `GET /demandas`: listar as demandas com, no mínimo, título, tipo, prioridade, status, projeto, responsável, data de criação e prazo de finalização;
- permitir filtragem por, no mínimo, dois critérios entre status, prioridade, tipo, responsável e projeto, utilizando parâmetros na URL (ex.: `/demandas?status=Aberta&prioridade=Alta`);
- permitir busca textual por título ou descrição;
- permitir ordenação por prioridade, data de criação, prazo de finalização ou status.

Aluno 4 – Cadastro de demanda e projetos

- `POST /demandas`: cadastrar uma nova demanda, validando os dados no servidor;
- exigir título, descrição, tipo, prioridade e projeto;
- aceitar apenas os tipos Tarefa, Defeito, Melhoria ou Documentação, e apenas as prioridades Crítica, Alta, Média ou Baixa;
- verificar se o projeto informado existe e, quando houver responsável informado, se o usuário existe (o responsável pode ficar em branco);
- validar o formato do prazo de finalização, quando informado;
- definir automaticamente, no servidor, o status Aberta, a data de criação e a data da última atualização, independentemente do que for enviado na requisição;

- retornar 201 com a demanda criada ou 400 com mensagem clara em caso de dados inválidos;
- `GET /projetos`: listar os projetos pré-cadastrados.

Aluno 5 – Detalhes e comentários

- `GET /demandas/:id`: retornar todos os campos de uma demanda específica e erro 404 quando ela não existir;
- `GET /demandas/:id/comentarios`: listar os comentários da demanda;
- `POST /demandas/:id/comentarios`: registrar um comentário obrigatoriamente vinculado à demanda e a um usuário, com data e horário registrados automaticamente pelo servidor;
- impedir comentários com texto vazio e retornar 404 quando a demanda não existir.

> **Atenção:** as validações feitas em JavaScript nas telas (Reunião 3) não substituem as validações no servidor. A API deve rejeitar dados inválidos mesmo que a requisição não venha das telas do sistema.

3. Teste das rotas

Nesta etapa, não é necessário que as telas consumam a API. As rotas deverão ser testadas diretamente:

- rotas GET: pelo navegador ou por ferramenta de teste de requisições;
- rotas POST: por ferramenta de teste de requisições, como Postman, Insomnia ou a extensão Thunder Client do VS Code.

Cada integrante deverá preparar requisições que demonstrem tanto situações válidas quanto situações de erro das suas rotas.

4. Relação com o escopo do PI2

As rotas devem permanecer alinhadas ao Sistema de Acompanhamento de Demandas de Desenvolvimento. Nesta etapa, considerar os seguintes elementos do escopo:

Elemento do sistema	O que deve ser atendido nesta etapa
Listagem de demandas	Retornar as informações principais de cada demanda (item 2.2.9 do Documento de Visão).
Filtros e buscas	Filtragem por no mínimo dois critérios, busca por título ou descrição e ordenação (item 2.3).
Dashboard	Totais e quantidades por status, prioridade e tipo, críticas em aberto e próximas do prazo (item 2.4).
Cadastro de demanda	Campos obrigatórios validados no servidor; status inicial Aberta e datas automáticas (itens 2.2.3 e 2.2.5).
Tipo e prioridade	Aceitar apenas os valores previstos: Tarefa, Defeito, Melhoria ou Documentação; Crítica, Alta, Média ou Baixa.
Comentários	Vinculados à demanda e ao usuário, com data e horário registrados (item 2.2.6).

Elemento do sistema	O que deve ser atendido nesta etapa
Autenticação	Versão inicial: conferência de e-mail e senha com retorno do perfil do usuário. O controle de permissões por perfil será tratado posteriormente.
Exclusão de demandas	Não criar rota de exclusão (DELETE). A exclusão física não é permitida; demandas serão canceladas em etapa posterior (item 2.2.8).

5. Limites desta etapa

Nesta reunião, não será exigido: conexão com o banco de dados, integração entre as telas e a API, edição de demandas, alteração de status, responsável ou prioridade, regras de transição do ciclo de vida, cancelamento, histórico de alterações, controle de permissões por perfil e verificação de feriado por API externa. Esses itens serão tratados nas próximas etapas.

6. Organização no GitHub

- Cada atividade desta etapa deverá estar registrada no GitHub Projects.
- Cada integrante deverá possuir tarefa(s) compatíveis com o que efetivamente desenvolveu.
- Registrar estimativa e esforço real das atividades executadas.
- Utilizar branches de desenvolvimento para as alterações realizadas e integrar o código somente após verificação.
- Realizar commits que permitam identificar a evolução do trabalho e a participação de cada integrante.
- Cada arquivo de código deverá conter identificação do autor responsável e comentários explicativos, conforme as regras do PI2.

7. O que deverá ser apresentado na Reunião 4

- o servidor Express com TypeScript iniciando e executando localmente;
- a organização das pastas e arquivos do backend e os dados de teste em memória;
- as rotas de cada integrante funcionando, com demonstração de requisições válidas e das respostas obtidas;
- ao menos uma situação de erro por integrante, com o código HTTP e a mensagem retornados;
- os filtros, a busca e a ordenação da listagem de demandas;
- a organização do código no repositório GitHub, as branches utilizadas e o histórico de commits;
- os cards e registros de esforço no GitHub Projects;
- a contribuição individual de cada integrante desde a reunião anterior.